

Per-Object Causality Metadata Under Churn: A Minimal Study of Version Vectors, Dotted Version Vectors, and Pruning

Distributed Computing Project Report

Abstract

Eventually consistent key-value stores use per-object causal metadata to decide whether versions are ordered or concurrent. This metadata must be precise enough to avoid losing conflicts, but small enough to attach to every stored version. This paper presents a minimal simulation study of version vectors (VV), dotted version vectors (DVV), fixed lease-pruned DVV, membership-aware lease-pruned DVV, and a coarse vnode-level VV baseline. The simulator intentionally models only the mechanisms needed to expose clock behavior: partial replication, delayed delivery, client sessions, hot-key contention, sibling sets, and churn. Across the main workload, exact DVV preserves the precision and recall of exact VV while reducing metadata by 51.7–60.1%. Stress configurations amplify this result: with 512 clients, VV grows to 331.3 bytes in the stable profile while DVV remains near 120.9 bytes; under hotspot churn, VV reaches 882–935 bytes while DVV remains 193–220 bytes. Approximate designs expose the tradeoff space. Coarse vnode VV is compact but semantically lossy, with precision around 0.83–0.85 and missed-conflict rates around 19–23%. Lease-DVV reduces metadata but forgets ancestry and creates stale siblings. Membership-aware lease-DVV avoids unnecessary stable-profile pruning, but under churn it shows that membership alone is not a safe substitute for causal stability. The main conclusion is that DVV is the strongest exact representation in this study, while metadata garbage collection requires additional protocol evidence beyond time or membership.

1 Introduction and Project Idea

Replicated key-value stores often accept writes at multiple replicas and reconcile later. When two writes update the same key, the system must decide whether one version causally descends from the other or whether both versions are concurrent siblings. This decision is made using metadata attached to each version. The central systems tension is simple: causal metadata should be exact, but exact metadata can become large on hot objects written by many clients.

This project studies that tension at the level of a single object store abstraction. The goal is not to introduce a new clock or reproduce a complete production database. Instead, the goal is to isolate how clock representation and actor granularity affect metadata size and semantic fidelity under motivated stressors such as many clients, hot-key contention, partial replication, and churn.

The study compares five clock families. Exact VV uses client actors and provides a correctness baseline. Exact DVV uses replica-issued dots and a compact summary, preserving exactness while reducing dependence on client cardinality. Fixed lease-DVV prunes expired causal context and therefore trades metadata for recall. Membership-aware lease-DVV is a production-inspired refinement that keeps active replica actors and starts expiration only after a replica leaves. Finally, vnode VV uses coarse replica actors without dots; it is included as a compact but intentionally lossy actor-granularity baseline.

This paper makes four contributions:

1. It presents a minimal simulator that separates clock-visible causal context from oracle true history, allowing direct measurement of clock precision, recall, stale siblings, and missed conflicts.
2. It gives an apples-to-apples comparison of exact client-actor VV and exact replica-dot DVV under bounded client sessions.
3. It demonstrates experimentally that coarse replica/vnode vectors are not semantically equivalent to exact vectors: they reduce metadata by inventing false ancestry.
4. It evaluates fixed and membership-aware pruning, showing that time and membership are useful knobs but insufficient for safe causal garbage collection without stability evidence.

2 Motivation

The motivation is driven by three systems observations.

2.1 Per-object metadata grows in the hard cases

The cost of causal metadata is most visible on hot keys and under large writer populations. A workload with only a few clients may hide the difference between clock designs, while a hot object written by hundreds of clients exposes whether metadata grows with client cardinality. Motivated extreme configurations are therefore useful: they are not intended to be average production traces, but to isolate the mechanism under study.

Table 1: Motivated configurations used to expose specific clock failure modes.

Configuration	Stressor	Motivation
High client count	Many logical writers	Tests VV actor growth
Hotspot churn	Hot key plus node churn	Stresses siblings and stale actors
Sparse replication	Incomplete propagation	Stresses stale local state
Burst churn	Mass leave/join periods	Tests departed actor metadata
Lease ablation	L8/L16/L32 pruning	Exposes metadata/recall curve

2.2 Actor granularity affects correctness

A vector entry represents one actor. If each client is an actor, exact VV can distinguish independent client writes but may grow with the number of clients. If each replica is an actor, metadata is smaller, but independent writes issued through the same replica can be falsely ordered unless the representation includes per-event dots. DVV addresses this by using replica actors for compactness while preserving individual write identity through dots.

The simulator includes a deterministic same-replica example: two independent clients write through the same replica. A coarse vnode VV may order these writes because both increment the same replica actor. DVV keeps distinct dots and correctly reports concurrency. This example motivates why the comparison includes both exact DVV and coarse vnode VV.

2.3 Garbage collection needs evidence

Production systems often want to remove old causal metadata. Fixed leases are a simple approximation: after a timeout, old actor entries are pruned. Membership-aware pruning is more realistic: active actors are retained and only departed actors are eligible for expiration. However, neither time nor membership proves that all relevant versions have propagated. The evaluation tests whether these simple policies are sufficient and shows where they fail.

3 Related Work

This project builds on a long line of work on logical time, optimistic replication, causal consistency, and metadata compaction. The study is not claiming DVV as a new clock; rather, it uses DVV and related designs to experimentally separate representation costs from pruning and actor-granularity effects.

3.1 Logical time and vector clocks

Lamport clocks introduced logical time as a way to assign timestamps consistent with the happens-before relation, but scalar Lamport timestamps cannot distinguish concurrency from ordering [1]. Fidge and Mattern independently developed vector clocks, which encode enough information to determine whether two events are ordered or concurrent in an asynchronous distributed computation [2, 3]. Charron-Bost later showed lower bounds on the size of logical clocks needed to characterize causality, making explicit that exact causality tracking has an inherent dimension cost [4]. Singhal and Kshemkalyani proposed optimizations for vector clocks that reduce communication overhead by sending only changed entries, but the logical representation remains tied to the set of actors [5].

The exact VV baseline in this paper follows this tradition: it treats client sessions as logical actors and therefore preserves precise per-client causality. The downside observed in the evaluation—metadata growth with writer cardinality—is the classic vector-clock cost in a per-object storage setting.

3.2 Version vectors and optimistic replication

Version vectors adapt vector-clock ideas to replicated objects. Parker et al. used version vectors to detect mutual inconsistency in replicated distributed files [6]. Subsequent optimistic replication systems, including Bayou, emphasized disconnected operation, application-level conflict resolution, and session guarantees [7, 8]. These systems established the pattern used by many eventually consistent stores: accept writes optimistically, preserve concurrent versions, and later reconcile them.

Dynamo is the most direct systems ancestor for this project. Dynamo uses consistent hashing, sloppy quorums, hinted handoff, and vector-clock metadata to expose conflicting object versions to applications as siblings [9]. Dynamo also describes practical truncation of vector clocks to bound metadata, acknowledging that truncation can affect reconciliation semantics. Riak, a Dynamo-inspired store, further exposed the operational importance of causal context and sibling management [10]. This project abstracts away Dynamo’s full storage stack and focuses on the clock behavior behind this sibling decision.

3.3 Dotted version vectors and causal contexts

Dotted version vectors were proposed to solve a specific problem in version-vector-based stores: representing individual version events precisely while keeping a compact causal summary [11, 12]. A dot names the unique event that created a version, while the version vector summarizes its causal past. This separation lets a store distinguish concurrent writes even when they are issued by the same replica actor. DVV was later evaluated in Riak, where it addressed practical sibling explosion and causality-tracking issues in a production-oriented key-value store [13].

This paper’s DVV result is consistent with that literature. The contribution is not the invention of DVV, but a controlled comparison against exact client-actor VV, fixed lease pruning, membership-aware pruning, and coarse vnode vectors using the same oracle history and workload events.

3.4 Compact and approximate causality metadata

Several systems reduce metadata by changing the representation or by accepting approximation. Interval Tree Clocks support dynamic process creation and retirement without fixed-size vectors, making them attractive when actor identities change over time [14]. Version vectors with exceptions and related causal-context representations encode gaps and non-contiguous histories more compactly than plain vectors [15]. Probabilistic clocks such as Bloom clocks trade exactness for bounded metadata and can introduce false causality relationships [16]. These designs are related to the vnode VV and lease-DVV results in this paper: compactness is useful, but the exact semantic failure mode matters. A clock that invents ancestry may drop real conflicts, while a clock that forgets ancestry may retain stale siblings.

3.5 Causal consistency and dependency tracking in storage systems

A separate line of work enforces causal consistency across keys and datacenters. Causal memory formalized the programming model and implementation problem for systems that preserve causal order without requiring total order [17]. COPS tracks and checks causal dependencies to provide scalable causal consistency in wide-area storage [19]; Eiger extends these ideas to richer transactions [20]. GentleRain uses physical clocks and a global stable time to reduce dependency-checking overhead [21]. Orbe and related systems explore scalable causal consistency protocols using dependency matrices and datacenter-level metadata [22]. PRACTI replication studies how to trade precision, consistency, and topology in practical replicated systems [18]. These systems target system-wide causal consistency or replication protocols rather than only per-object sibling detection, but they motivate an important point for this project: production systems often need additional propagation or stability information beyond a local clock stamp.

3.6 Causal stability and metadata garbage collection

CRDT and causal-consistency research distinguishes causality tracking from causal stability. Shapiro et al. survey CRDT designs that rely on causal delivery, version vectors, and metadata compaction to support convergence [23]. Later work on causal stability shows how metadata can be reduced once the system knows that relevant updates have been observed by all replicas that need them [24]. This idea directly explains the membership-aware lease-DVV result in this paper. Membership says whether an actor is currently active, but it does not prove that the actor’s prior writes have been delivered, merged, or made stable. Safe pruning needs a stability signal, such as handoff completion, acknowledgments, or anti-entropy evidence.

3.7 Positioning of this study

Compared with prior work, this project is deliberately narrower. It does not propose a new causal consistency protocol, a new production store, or a new logical clock. Instead, it provides a minimal experimental framework for asking: given the same per-object read context, how do different clock representations trade metadata size for semantic fidelity? This positioning is why the simulator includes oracle true histories, exact VV and DVV baselines, lossy vnode vectors, and pruning variants in the same workload matrix.

4 Design

The simulator models a minimal eventually consistent per-object store. It includes the mechanisms needed to evaluate causal clocks and omits mechanisms orthogonal to the clock representation, such as anti-entropy, Merkle trees, disk layout, request routing, and real membership handoff.

4.1 Store semantics

A cluster contains active nodes. A client chooses a key and coordinator. The coordinator reads its local sibling set for that key, combines it with the client’s session context, issues a clock stamp, stores the new version, and asynchronously replicates it to a subset of active nodes. Each node stores a sibling set per key. When a version arrives, the node compares the incoming stamp with each local sibling. Dominated versions are dropped; concurrent versions are retained.

Every version also carries an oracle true-history set. This set is not visible to the clock. It is used only by the evaluator to determine whether the clock’s represented ancestry matches the real dependency history generated by reads and writes.

4.2 Clock models

VV stores a prefix map from client actor to counter. It is exact with respect to the simulator’s client-session semantics, but grows as more client actors write the same object. **DVV** stores a causal summary plus a new dot (*actor, counter*) for the current write. It is exact in this simulator because the dot preserves event identity even when actors are replicas. **Lease-DVV** starts from DVV and prunes expired actors/events before issuing a new write. **Membership-aware Lease-DVV** keeps active replica actors and starts the lease timer only after a replica leaves. **VNode VV** stores coarse replica counters and is evaluated as a lossy baseline.

4.3 Metrics

The main metrics are defined relative to oracle true history. Precision is the fraction of represented events that are true ancestors. Recall is the fraction of true ancestors represented by the stamp. A missed conflict occurs when the oracle says two versions are concurrent but the clock orders them. A stale sibling occurs when the oracle says one sibling dominates another but the clock keeps both. Metadata bytes are an approximate, consistent encoding cost used for relative comparison across clocks.

4.4 Validity and abstraction boundary

The simulator is deliberately small. It does not claim to predict end-to-end production performance. Instead, it provides controlled experiments where all clocks see the same workload events and read contexts. Omitting anti-entropy is acceptable for the main question because anti-entropy changes propagation and convergence, while this study asks whether a given stamp representation can distinguish ancestry and concurrency from the same observed context. Anti-entropy and handoff would matter for safe pruning; this is why causal-stability-based garbage collection is treated as future work rather than assumed.

5 Implementation

The implementation is a Python package organized into modules for clocks, causal context, store semantics, simulation, metrics, and experiment orchestration. Each clock implements the same interface: construct a read context, issue a stamp, compare stamps, observe stamps on delivery, and estimate metadata size. This common interface ensures that differences in results come from clock semantics rather than different store behavior.

The most important implementation detail is the separation between clock-visible metadata and oracle true history. On every write, the simulator constructs the true history by unioning the histories of all read versions and adding the new event. The clock receives only the versions’ stamps. This allows the evaluator to measure false positives, false negatives, stale siblings, and missed conflicts directly.

Experiments are configured in YAML and executed through reproducibility scripts. Outputs are organized into aggregate CSVs, figures, time-series plots, configuration snapshots, and per-run raw data. The primary run used in this report is:

`output/experiments/2026-05-03_02-21-48_per_object_clock_study_final`

The membership-aware pruning run is:

`output/experiments/2026-05-03_23-37-00_membership_lease_study`

The codebase is validated by 27 tests covering context semantics, exact clock behavior, lease pruning, membership-aware pruning, and scenario smoke tests.

6 Evaluation

6.1 Exact DVV preserves correctness with lower metadata

Table 2 gives the main exact-clock result. VV and DVV both achieve precision and recall of 1.0 in every profile. DVV, however, uses roughly half the metadata of VV.

Table 2: Exact-clock comparison. DVV preserves exactness while reducing metadata relative to VV.

Profile	VV bytes	DVV bytes	Reduction
Stable	294.7	123.4	58.1%
Low churn	318.1	126.8	60.1%
Sustained churn	308.9	139.2	54.9%
Burst churn	312.7	151.1	51.7%

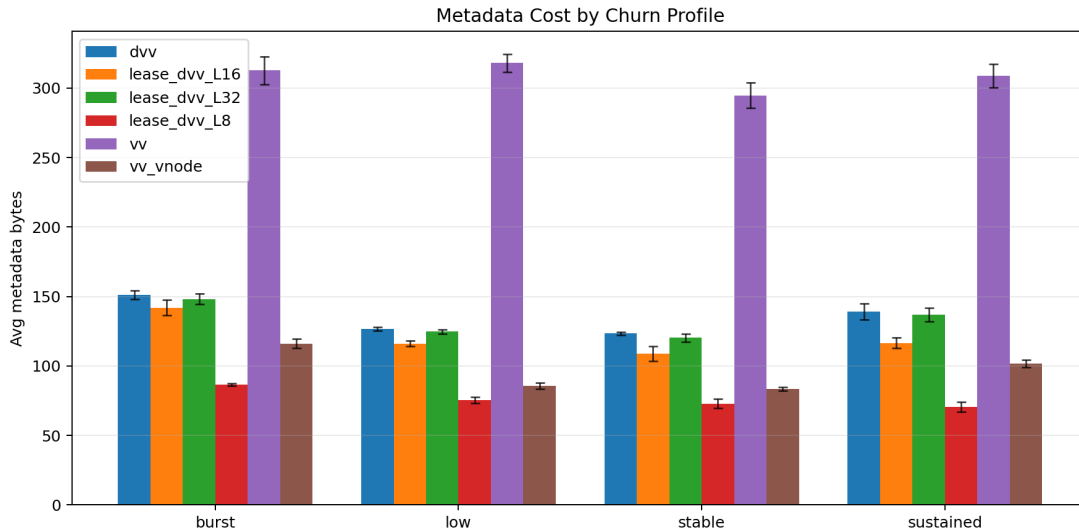


Figure 1: Average metadata bytes by profile. Exact DVV is consistently smaller than exact VV while preserving perfect precision and recall.

The sensitivity and stress runs make the scaling effect more visible. With 32 clients in the stable profile, VV uses 186.4 bytes and DVV uses 122.1 bytes. With 512 clients, VV grows to 331.3 bytes while DVV remains 120.9 bytes. In extreme hotspot churn, VV reaches 882–935 bytes while DVV remains 193–220 bytes. These results support the claim that DVV’s benefit is largest when many logical writers touch the same object.

6.2 VNode VV is compact but loses semantic precision

VNode VV is smaller than exact VV, but it achieves compactness by using coarse replica actors. In the main run, vnode VV has recall 1.0 but precision around 0.83–0.85 and missed-conflict rates around 19–23%. This means it tends to invent ancestry: two concurrent writes can appear ordered if they share the same coarse actor. This is a dangerous failure mode because it can drop real conflicts. The result shows why vnode VV should be treated as a lossy actor-granularity baseline, not as an exact VV implementation.

6.3 Lease-DVV exposes the pruning tradeoff

Fixed lease-DVV shows a clear metadata/recall curve. Short leases prune aggressively and save the most metadata, but they forget ancestry and leave stale siblings. Long leases approach exact DVV behavior but provide less metadata reduction.

Table 3: Fixed lease-DVV tradeoff. Pruning reduces metadata but lowers ancestry recall and increases stale sibling retention.

Profile	Clock	Bytes	Recall	Stale sibling rate
Stable	DVV	123.4	1.000	0.000
Stable	Lease L8	72.9	0.528	0.839
Stable	Lease L16	108.9	0.876	0.253
Stable	Lease L32	120.3	0.972	0.045
Sustained	DVV	139.2	1.000	0.000
Sustained	Lease L8	70.5	0.451	0.839
Sustained	Lease L16	116.6	0.821	0.377
Sustained	Lease L32	136.8	0.974	0.097

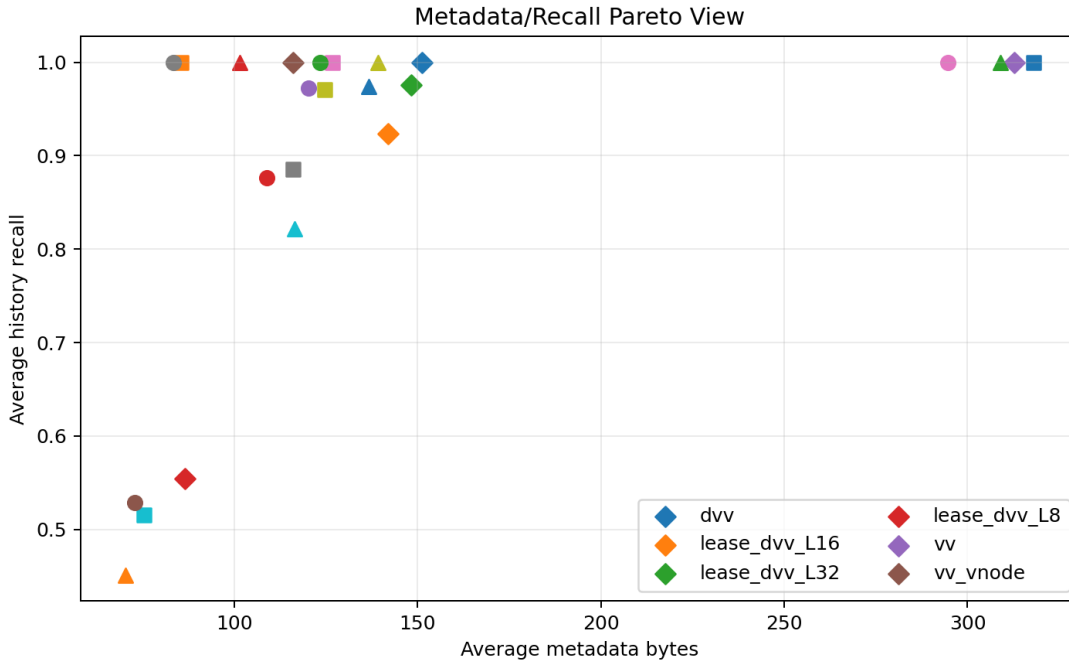


Figure 2: Metadata/recall Pareto frontier. Exact DVV sits at perfect recall with lower metadata than VV. Lease-DVV moves left by giving up recall.

This is not a failure of DVV. It is the expected cost of approximate garbage collection: once ancestry is forgotten, the clock may no longer prove that an old sibling is dominated.

6.4 Membership-aware pruning improves stable behavior but not churn safety

Membership-aware lease-DVV tests whether membership is a sufficient production-inspired signal for pruning. In the stable profile, it works exactly as intended. Membership-aware L16 matches exact DVV: 121.7 bytes, recall 1.0, and stale sibling rate 0.0. Fixed L16 in the same run has 110.0 bytes, recall 0.902, and stale sibling rate 0.242. This confirms that fixed leases unnecessarily prune quiet but active actors.

Under heavy churn, however, membership-aware pruning is not enough. In burst churn, membership-aware L16 reduces metadata to 97.4 bytes, but recall falls to 0.668 and stale sibling rate rises to 0.867. The reason is conceptual rather than accidental: membership says whether a replica is active, but not whether all causally relevant writes from a departed actor have been propagated or merged. Safe pruning requires stronger evidence, such as handoff completion, anti-entropy acknowledgments, or causal stability.

7 Limitations

This study is a simulator-based mechanism study, not a deployment benchmark. The metadata byte model is approximate, although applied consistently across clocks. The workload distributions are synthetic and intentionally stressful. The simulator does not include background anti-entropy, read repair, real membership handoff, network partitions beyond delay and churn, application-specific merge functions, throughput measurements, or storage-engine effects. These omissions are deliberate: they keep the experiment focused on per-object clock representation. They also define the boundary of the conclusions. In particular, the pruning results should be read as evidence that simple time and membership policies are insufficient, not as a complete evaluation of production garbage collection.

8 Conclusion

The main result is that exact DVV is the strongest exact representation in this study. It preserves the semantic correctness of exact VV while reducing metadata by 51.7–60.1% in the main workload and by more in motivated stress configurations. Coarse vnode VV demonstrates that compact actor granularity can be semantically dangerous because it creates false ancestry and misses conflicts. Lease-DVV demonstrates the other side of the design space: pruning can reduce metadata, but it is a metadata/correctness knob rather than a free optimization. Membership-aware lease-DVV improves stable-case behavior, but its churn failures show that membership is not causal stability.

The design lesson is to separate representation from garbage collection. DVV is a good exact per-object representation. Safe pruning should be layered on top only when the system has additional evidence that old causal information is stable, such as completed handoff or anti-entropy acknowledgments.

References

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [2] C. J. Fidge, “Timestamps in message-passing systems that preserve the partial ordering,” in *Proceedings of the 11th Australian Computer Science Conference*, 1988.
- [3] F. Mattern, “Virtual time and global states of distributed systems,” in *Parallel and Distributed Algorithms*, North-Holland, 1989.
- [4] B. Charron-Bost, “Concerning the size of logical clocks in distributed systems,” *Information Processing Letters*, vol. 39, no. 1, pp. 11–16, 1991.
- [5] M. Singhal and A. Kshemkalyani, “An efficient implementation of vector clocks,” *Information Processing Letters*, vol. 43, no. 1, pp. 47–52, 1992.
- [6] D. S. Parker, G. J. Popek, G. Rudisin, A. Stoughton, B. J. Walker, E. Walton, J. M. Chow, D. Edwards, S. Kiser, and C. Kline, “Detection of mutual inconsistency in distributed systems,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 3, pp. 240–247, 1983.
- [7] A. Demers, K. Petersen, M. Spreitzer, D. Terry, M. Theimer, and B. Welch, “The Bayou architecture: Support for data sharing among mobile users,” in *Proceedings of the IEEE Workshop on Mobile Computing Systems and Applications*, 1994.

- [8] D. B. Terry, A. J. Demers, K. Petersen, M. J. Spreitzer, M. M. Theimer, and B. B. Welch, “Session guarantees for weakly consistent replicated data,” in *Proceedings of the International Conference on Parallel and Distributed Information Systems*, 1994.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.
- [10] Basho Technologies, “Riak KV causal context,” Riak documentation. Available: <https://docs.riak.com/riak/kv/latest/learn/concepts/causal-context/>.
- [11] P. S. Almeida, C. Baquero, and V. Fonte, “Dotted version vectors: Logical clocks for optimistic replication,” arXiv:1011.5808, 2010.
- [12] P. S. Almeida, C. Baquero, R. Goncalves, N. Pregoica, and V. Fonte, “Dotted version vectors: Efficient causality tracking for distributed key-value stores,” in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC), Brief Announcement*, 2012.
- [13] R. Goncalves, P. S. Almeida, C. Baquero, and N. Pregoica, “Evaluating dotted version vectors in Riak,” technical report / evaluation study, Universidade do Minho and Universidade Nova de Lisboa, 2013.
- [14] P. S. Almeida, C. Baquero, and V. Fonte, “Interval tree clocks,” in *Proceedings of the 12th International Conference on Principles of Distributed Systems (OPODIS)*, 2008.
- [15] N. Pregoica, C. Baquero, P. S. Almeida, V. Fonte, and R. Goncalves, “Brief announcement: Efficient causality tracking in distributed storage systems with dotted version vectors,” in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2012.
- [16] A. D. Kshemkalyani and N. Misra, “The Bloom clock to characterize causality in distributed systems,” arXiv:1905.13064, 2019.
- [17] M. Ahamad, G. Neiger, J. E. Burns, P. Kohli, and P. W. Hutto, “Causal memory: Definitions, implementation, and programming,” *Distributed Computing*, vol. 9, no. 1, pp. 37–49, 1995.
- [18] N. Belaramani, M. Dahlin, L. Gao, A. Nayate, A. Venkataramani, P. Yalagandula, and J. Zheng, “PRACTI replication,” in *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006.
- [19] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen, “Don’t settle for eventual: Scalable causal consistency for wide-area storage with COPS,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, 2011.
- [20] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen, “Stronger semantics for low-latency geo-replicated storage,” in *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013.
- [21] J. Du, C. Iorgulescu, A. Roy, and W. Zwaenepoel, “GentleRain: Cheap and scalable causal consistency with physical clocks,” in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2014.
- [22] J. Du, S. Elnikety, and W. Zwaenepoel, “Orbe: Scalable causal consistency using dependency matrices and physical clocks,” in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2013.
- [23] M. Shapiro, N. Pregoica, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, 2011.
- [24] J. Bauwens and E. Gonzalez Boix, “From causality to stability: Understanding and reducing metadata in CRDTs,” in *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, 2020.